TP Sécurité des Smartphones
SecureNotesLab — Challenge final

Étudiante : Chaymae Noughou
Date : 10/10/2026

Objectif : analyser une version piégée de SecureNotesLab, fournie sans indication du nombre ni de l'emplacement des failles, puis démontrer chaque faiblesse avec AttackLab et ADB, la corriger et vérifier la correction.

Applications : SecureNotesLab (com.lab.securenotes), version Challenge volontairement vulnérable, et AttackLab (com.lab.attacklab), application tierce de test munie d'un bouton par faille. Données fictives, périmètre strictement limité au laboratoire.

Méthode : Observer → Attaquer → Comprendre → Corriger → Retester.

Environnement : émulateur Pixel 8, Android 15 (API 35), Android Studio, ADB.

1. Démarche et failles identifiées

L'analyse du code source (manifeste, activités, ContentProvider) a permis d'identifier sept points fragiles, répartis sur plusieurs fichiers. Chacun a ensuite été démontré depuis AttackLab, qui comporte un bouton par faille. Les failles sont notées C1 à C7 et détaillées ci-dessous.

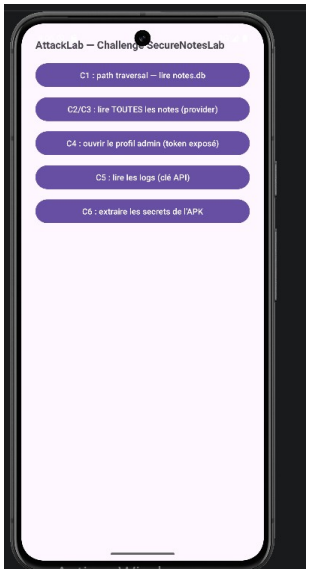


Figure 1 – AttackLab : un bouton de test par faille du Challenge.

2. C1 — Path traversal (lecture de fichier arbitraire)

Faiblesse : NoteDetailActivity est exportée et lit File(filesDir, "notes/$nomFichier") avec un nomFichier venu de l'Intent, sans le valider.

Test réalisé : AttackLab envoie cheminFichier = "../../databases/notes.db".

```
intent.putExtra("cheminFichier", "../../databases/notes.db")
```



Figure 2 – NoteDetailActivity lit la base notes.db : la note de bob « Code d acces entrepot : 4471 » apparaît en clair.

Interprétation : les « ../ » permettent de sortir du dossier notes et d'atteindre n'importe quel fichier du sandbox (ici la base SQLite). Une entrée externe utilisée comme chemin de fichier, sans contrôle, donne une lecture arbitraire.

Correction : n'accepter qu'un nom de fichier simple et vérifier que le chemin canonique reste dans le dossier notes.

```
val nom = intent.getStringExtra("cheminFichier") ?: ""
if (nom.contains("/") || nom.contains("..")) { /* refuser */ }
val base = File(filesDir, "notes").canonicalPath
val f = File(filesDir, "notes/$nom")
if (!f.canonicalPath.startsWith(base)) { /* refuser */ }
```

3. C2 / C3 — ContentProvider : contrôle d'accès cassé et permission faible

Faiblesse : le ContentProvider NotesProvider est exporté et protégé par la permission READ_NOTES en protectionLevel="normal" (C3). De plus, query() ignore la sélection et fait db.query("notes", …, null, null, …), renvoyant toute la table (C2).

Test réalisé : AttackLab déclare simplement la permission (auto-accordée car « normal ») puis interroge le provider.

```
contentResolver.query(Uri.parse("content://com.lab.securenotes.provider"), null, null,
null, null)
```

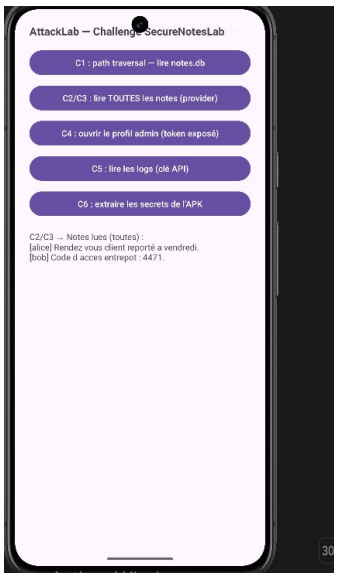


Figure 3 – AttackLab lit TOUTES les notes, y compris celle de bob (« Code d acces entrepot : 4471 »).

Interprétation : une permission « normal » est accordée automatiquement à toute application qui la déclare : elle ne protège donc rien contre une appli tierce. Et même protégé, le provider renvoie les données de tous les utilisateurs car la requête ne filtre pas. Permission technique ≠ autorisation métier.

Correction : passer la permission en protectionLevel="signature" (C3) et filtrer la requête sur l'utilisateur authentifié (C2).

```
<permission android:name="...READ_NOTES" android:protectionLevel="signature" />

// dans query() : ne renvoyer que les notes de l'utilisateur courant
db.query("notes", projection, "proprietaire = ?", arrayOf(utilisateurCourant), null,
null, sortOrder)
```

4. C4 — Activity sensible exportée (profil + token)

Faiblesse : ProfilActivity est exportée et affiche le token de session et « Niveau : administrateur », sans vérifier Session.estConnecte.

Test réalisé : AttackLab lance directement ProfilActivity.

```
intent.setClassName("com.lab.securenotes", "com.lab.securenotes.ProfilActivity")
```

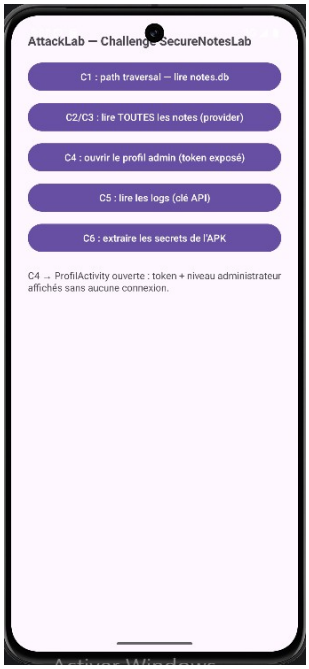


Figure 4 – AttackLab ouvre ProfilActivity : token et niveau administrateur exposés sans aucune connexion.

Interprétation : un écran affichant des données sensibles est ouvert par une application étrangère, sans authentification. Le token exposé est demo_token_C7F2 (confirmé aussi par l'extraction de l'APK, faille C6).

Correction : passer ProfilActivity en exported="false", vérifier la session à l'entrée, et ne pas afficher le token à l'écran.

```
<activity android:name=".ProfilActivity" android:exported="false" />

if (!Session.estConnecte) { finish(); return }
```

5. C5 — Donnée sensible dans les logs

Faiblesse : NotesListActivity et SyncManager journalisent la clé d'API : Log.d("SecureNotesLab", "... sync key = ${SyncManager.API_KEY}").

Test réalisé : AttackLab tente de lire les journaux.



Figure 5 – Sur Android moderne, une appli tierce ne lit que ses propres logs ; la clé reste néanmoins visible via « adb logcat ».

Interprétation : une application tierce ordinaire ne lit pas librement les journaux des autres applications. Mais la clé journalisée reste exposée dans tout contexte où les logs sont collectés (débogage, diagnostic, appareil compromis). On ne journalise jamais un secret.

Correction : ne pas inclure la clé dans le message de log.

```
Log.d("SecureNotesLab", "Synchronisation lancée")
```

6. C6 — Secrets codés en dur

Faiblesse : SyncManager.API_KEY = "SNL-SYNC-98213" et Session.token = "demo_token_C7F2" sont écrits en dur dans le code.

Test réalisé : AttackLab lit l'APK installé de SecureNotesLab et y recherche ces chaînes.

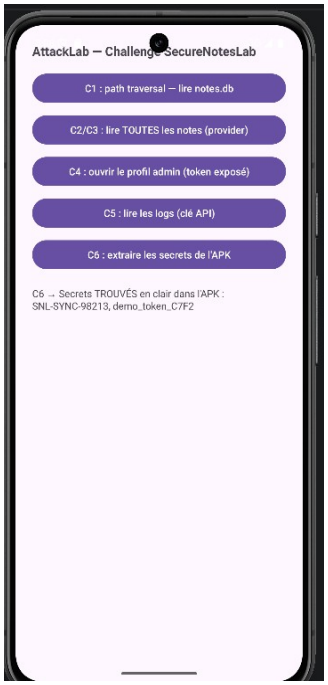


Figure 6 – Secrets retrouvés en clair dans l'APK : SNL-SYNC-98213 et demo_token_C7F2.

Interprétation : compiler ne rend pas un secret confidentiel : les chaînes restent lisibles dans l'artefact. Un secret serveur livré avec l'application est récupérable, quel que soit l'endroit où il est ensuite stocké.

Correction : ne pas livrer de secret serveur dans l'application ; obtenir la valeur dynamiquement (après authentification), sans l'écrire dans le binaire.

7. C7 — Sauvegarde autorisée

Faiblesse : le manifeste déclare android:allowBackup="true".

Interprétation : les données privées de l'application (notes, base) peuvent être incluses dans une sauvegarde (adb backup) et extraites hors de l'appareil. (Faille de configuration ; le mécanisme adb backup est restreint sur Android récent, mais le réglage reste à éviter.)

Correction : passer android:allowBackup="false".

```
<application android:allowBackup="false" ... >
```

8. Synthèse

Faille	Emplacement	Preuve (démonstration)	Correction
C1 — Path traversal	NoteDetailActivity (exportée), cheminFichier non validé	AttackLab fait lire ../../databases/notes.db → note de bob en clair	Valider le nom (pas de / ni ..), rester dans le dossier notes
C2 — Provider : accès cassé	NotesProvider.query() ignore la sélection	Lecture de TOUTES les notes (alice + bob)	Filtrer par utilisateur authentifié + honorer la sélection
C3 — Permission faible	READ_NOTES en protectionLevel=normal	Permission auto-accordée à AttackLab	protectionLevel=signature
C4 — Activity exportée	ProfilActivity exportée, sans contrôle	ProfilActivity ouverte : token + admin sans connexion	exported=false + vérifier la session, ne pas afficher le token
C5 — Logs sensibles	NotesListActivity / SyncManager	Clé API journalisée (visible via adb logcat)	Ne pas journaliser la clé
C6 — Secrets codés en dur	SyncManager.API_KEY, Session.token	Secrets extraits de l'APK par AttackLab	Secret obtenu dynamiquement, hors binaire
C7 — Sauvegarde autorisée	allowBackup=true (manifeste)	Données incluses dans une sauvegarde (adb backup)	allowBackup=false

Conclusion

La version Challenge combinait sept faiblesses de conception, toutes exploitées depuis une application tierce par des canaux que SecureNotesLab avait elle-même ouverts : composant exporté sans contrôle (C1, C4), entrée non validée (C1), contrôle d'accès cassé et permission mal choisie (C2, C3), secret journalisé (C5), secret codé en dur (C6), sauvegarde autorisée (C7). Aucune n'a nécessité de casser la sandbox : c'est le code et la configuration de l'application qui ont cédé. Chaque faille a une correction ciblée — validation d'entrée, permission signature, contrôle métier, non-journalisation et non-stockage des secrets — à vérifier en rejouant exactement la même attaque.